

ARGUS Detection Engine

A Code-Driven Engineering Analysis

ARGUS Engineering
<https://arguslabs.dev>

May 2026 — Revision 2

Abstract

This document explains, from the ground up, how the ARGUS detection engine decides whether a node is healthy, degraded, or critical. It is written for engineers who need to maintain, debug, or redesign the system. Every claim is traceable to the implementation. Where the code is incomplete or behaves in a way that may surprise an operator, that is stated explicitly.

The document is structured so that each section builds on the previous one. A reader unfamiliar with ARGUS should be able to read it sequentially and arrive at a complete understanding of the engine's behavior.

Contents

1	System Overview	4
1.1	What problem does the detection engine solve?	4
1.2	Why is this hard?	4
1.3	Outputs	4
1.4	Where the engine fits in the agent	5
2	Codebase Mapping	5
2.1	How the code is organized	5
2.2	The per-window execution flow	5
3	Data Flow Pipeline	7
3.1	Three data sources	7
3.2	The aggregator	7
3.3	Window timing and its implications	8
4	Detection Logic	8
4.1	Stage 1: Composite raw score	8
4.2	Stage 2: IB-aware carry-forward	9
4.3	Stage 3: Capability fusion	10
4.4	Stage 4: Coverage-weighted severity floor	11
4.5	Putting it all together: one evaluation cycle	11
5	Smoothing: From Raw Score to Effective Score	12
5.1	Why smooth at all?	12
5.2	The dual-track smoother	12

6	The State Machine	13
6.1	Why a state machine?	13
6.2	The four states	13
6.3	Configuration defaults	14
6.4	Transition rules	14
6.5	Transition diagram	15
6.6	Worked example: 20% packet loss, then recovery	15
7	Detection Rules	16
7.1	Overview	16
7.2	Rule reference	16
7.3	Rule detail: InterruptAffinitySkew	16
7.4	Rule detail: LatencyDrift	17
7.5	Rule detail: ThroughputDrop	17
7.6	Rule detail: CqJitter	18
7.7	Rule detail: CongestionSpread	18
7.8	Rule detail: PcieBottleneck	18
7.9	Deep dive: RdmaLinkDegradation (the most complex rule)	18
7.10	RollingStats: the statistical backbone	19
8	Multi-Timescale Evaluation and Burst Classification	20
8.1	Why multiple timescales?	20
8.2	Burst classification	20
9	Timing Model	21
9.1	Time constants at a glance	21
9.2	Timing edge cases	21
10	Failure Modes	21
10.1	Recovery takes longer than expected	21
10.2	Carry-forward extends recovery when workload stops	22
10.3	Missing or incomplete sysfs counter layout	22
10.4	Capability fusion never reduces the score	22
10.5	Determinism with caveats	22
11	Observability and Debuggability	22
11.1	What is exported	22
11.2	What is not exported	23
11.3	How to trace a decision	23
12	Limitations and Gaps	24
12.1	Signals the engine cannot observe today	24
12.2	Assumption failures	24
12.3	Missing safeguards	24
13	Summary of Actual Behavior	24

14 eBPF Kernel Probes	25
14.1 Probe programs	25
14.2 BPF maps	25
14.3 CQ jitter measurement path	26
15 Hardware Counter Collection	27
15.1 Counter types	27
15.2 Derived metrics	27
16 Capability System	28
16.1 Design rationale	28
16.2 Capabilities	28
16.3 Quality levels and coverage grading	28
16.4 Provider backends	28
17 Scheduler Integration	29
17.1 Architecture	29
17.2 State model	29
17.3 Policy	29
17.4 Safety mechanisms	29
17.5 SLURM backend	30
18 Autonomous Actions	30
18.1 Action types	30
18.2 Rate limiting	30
19 Prometheus Metrics Reference	31
19.1 Health and detection	31
19.2 eBPF subsystem metrics	31
19.3 IB hardware counter metrics	31
19.4 Capability and timescale metrics	32
19.5 Scheduler metrics	32
20 Security Architecture	32
20.1 Privilege model	32
20.2 Systemd hardening	33
20.3 eBPF integrity	33
20.4 Metrics endpoint security	33
21 Configuration Reference	33
21.1 Detection tuning parameters	33
21.2 State machine parameters	34
21.3 Non-configurable rule parameters	34
21.4 Environment variables	34

1 System Overview

1.1 What problem does the detection engine solve?

ARGUS monitors InfiniBand and RDMA network health on individual compute nodes. Raw signals—interrupt counts, error counters, completion latencies, memory allocation behavior—arrive continuously from the kernel. These signals are noisy, bursty, and sometimes absent. The detection engine’s job is to turn this stream of noisy data into a single, stable answer:

Is this node healthy enough to run workloads right now?

The answer takes the form of one of four **health states**: **Healthy**, **Degraded**, **Critical**, or **Recovering**. This verdict can drive automated actions such as draining the node from SLURM or marking a Kubernetes node as unschedulable.

1.2 Why is this hard?

A naive approach—“if errors exceed a threshold, mark the node as unhealthy”—fails in production for three reasons:

1. **Noise.** Transient retransmissions and symbol errors occur on healthy InfiniBand links. A simple threshold fires constantly.
2. **Flapping.** Network impairments cause alternating windows of high and zero errors (e.g., RDMA backoff pauses traffic after errors). A threshold-based system oscillates between healthy and critical.
3. **Absent signals.** Different hardware exposes different counters. An IB HCA, a RoCE NIC, and a Soft-RoCE device each present different sysfs layouts. The engine must degrade gracefully when data is missing.

The detection engine addresses all three through layered smoothing, asymmetric hysteresis, and a capability-aware confidence system.

1.3 Outputs

The engine produces two things every evaluation cycle:

1. **Alerts.** A list of specific problems detected by individual rules (e.g., “IRQ affinity skew: CPU 0 handling 85% of interrupts”). Alerts are emitted every window a rule fires, regardless of whether the health state changes.
2. **Health state.** A single verdict: **Healthy**, **Degraded**, **Critical**, or **Recovering**. This is the authoritative signal consumed by the scheduler.

Key Insight

Alerts and health state are independent. A rule can fire alerts every window while the health state remains unchanged. The state only transitions after sustained evidence crosses smoothing and dwell thresholds.

1.4 Where the engine fits in the agent

Before reading further, the following terms appear throughout this document:

Definition

Window

A fixed-length time interval (default 3 seconds) during which the agent accumulates metrics. At the end of each window, the detection engine evaluates once.

Evaluation cycle

One execution of the detection engine at the end of a window. Synonymous with “one window tick” in the code.

Raw score

A number in $[0,1]$ computed from the current window’s metrics. 0 = perfectly healthy; 1 = maximally degraded.

Effective score

The raw score after smoothing (EWMA + peak-hold). This is what the state machine actually sees.

State transition

A change in the health verdict (e.g., Healthy $\rightarrow$ Degraded). Requires sustained evidence, not a single high score.

Dwell The number of consecutive windows that a condition must hold before a transition occurs.

The agent’s architecture, in one sentence: *eBPF probes and sysfs readers produce raw data, the aggregator summarizes it per-window, and the detection engine evaluates the summary.*

The next section maps this to the codebase.

2 Codebase Mapping

2.1 How the code is organized

The detection engine is not a single file. It is spread across several modules, each with a focused responsibility. Understanding which file owns which concern is essential for debugging.

2.2 The per-window execution flow

Every 3 seconds (by default), the following sequence runs. This is the single most important thing to understand about the engine’s control flow.

```
// === Phase 1: Data Collection (main.rs) ===
// Read eBPF maps: IRQ counts, slab alloc stats, NAPI polls, CQ jitter.
// These are kernel-side accumulators; the agent reads them once per window.
snap = ebpf_source.read_bpf_snapshot();
pipeline.ingest_bpf_snapshot(&snap);

// Read sysfs IB hardware counters (symbol_errors, link_downed, etc.)
// The aggregator computes deltas: current_value - previous_value.
```

File	What it owns
detection/mod.rs	The three core types: <code>DetectionEngine</code> , <code>SmoothedHealthScore</code> , and <code>HealthStateMachine</code> . Also contains the composite score formula.
detection/rules.rs	The <code>DetectionRule</code> trait and all 11 concrete rule implementations.
detection/fusion.rs	The function that converts capability provider samples into a score contribution.
detection/timescale.rs	Three parallel evaluation tracks (fast, medium, slow) for multi-timescale analysis.
detection/burst.rs	A classifier that distinguishes transient bursts from sustained degradation.
detection/rolling_stats.rs	EWMA tracker with clamping, z-score computation, and a trend (monotonic rise) detector.
pipeline/mod.rs	Orchestrates the per-window flow: collect data, poll providers, run detection.
pipeline/aggregator.rs	Accumulates raw events into <code>AggregatedMetrics</code> . Computes hardware counter deltas. Maintains a <code>DDSketch</code> for CQ latency quantiles.
main.rs	The tick loop. Controls window timing, reads BPF maps, reads sysfs counters, and wires results to the exporter and scheduler.

Table 1: File-level responsibilities in the detection subsystem.

```

for hw_event in hw_reader.read_all() {
    pipeline.ingest(&hw_event);
}

// === Phase 2: Detection (pipeline/mod.rs) ===
// Poll capability providers for additional signals (e.g., link errors,
// retransmit rates, peer liveness). Each emits zero or more Samples.
samples = capabilities.collect_all(&context);

// Run all 11 detection rules against the aggregated metrics.
// Compute composite score from metrics + samples + coverage report.
// Feed the smoothed score into the state machine.
alerts = detection.evaluate_with_coverage(metrics, &samples, &coverage);

// === Phase 3: Export (main.rs) ===
// Update Prometheus gauges, TUI state, scheduler reconciler.
pipeline.reset_window();           // Clear metrics for next window.
                                   // Detection state persists.

```

Listing 1: Annotated per-window execution flow

Key Insight

The aggregator resets every window (fresh metrics each cycle), but the detection engine’s state persists across windows. This is how it remembers baselines, maintains evidence counters, and resists flapping.

3 Data Flow Pipeline

This section traces how raw kernel events become the `AggregatedMetrics` struct that the detection engine evaluates.

3.1 Three data sources

The agent collects data from three independent sources, each read once per window:

1. eBPF maps. eBPF programs attached to kernel tracepoints maintain per-CPU counters in BPF map arrays. The agent reads these maps once per window via `read_bpf_snapshot()`. The snapshot includes:

- Per-CPU interrupt counts (IRQ distribution).
- Slab allocation count, byte totals, and latency sum.
- NAPI poll count, total work done, and total budget.
- CQ completion count, latency sum, max latency, and stall count (completions $> 50\mu\text{s}$).

These values overwrite the corresponding fields in `AggregatedMetrics` directly (they are per-window deltas computed in-kernel).

2. sysfs hardware counters. The agent reads files under `/sys/class/infiniband/*/ports/*/` once per window. Each file contains an absolute counter (e.g., total symbol errors since boot). The aggregator tracks the previous value in a `HashMap<(port, counter_type), u64>` and computes:

$$\text{delta} = \text{current.saturating_sub}(\text{previous})$$

This produces the `IbCounterDeltas` struct, which contains per-window deltas for 17 counter types including symbol errors, link downed events, receive/transmit data, and software-RDMA (RXE) counters where applicable.

3. Capability providers. In addition to the two raw-data paths, the engine polls a set of `CapabilityProvider` implementations (e.g., sysfs link error provider, throughput provider, inferred retransmit provider). Each emits zero or more `Sample` values annotated with a quality level (High, Medium, Low, or Absent). These flow into the fusion layer (Section 4.3).

3.2 The aggregator

The `Aggregator` (`pipeline/aggregator.rs`) is the bridge between raw events and detection. Its key properties:

- It accumulates metrics for one window in `AggregatedMetrics`.
- At `reset()`, all per-window fields are zeroed, but the previous-counter `HashMap` persists (so deltas remain correct in the next window). The `per_cpu_counts` vector is reallocated to `num_cpus` zeros.
- It maintains a `DdSketch` ($\alpha = 0.02$, max 1024 bins) for CQ completion latencies. This sketch provides approximate quantiles (p50, p95, p99, p999) within 2% relative error. It is also reset per window.

Hardware counter deltas. The aggregator maintains a `HashMap<(port, discriminant), u64>` of previously observed counter values. On each ingest, `delta = current.saturating_sub(previous)`. This delta is *added* to the relevant field in `IbCounterDeltas`—meaning deltas from multiple ports accumulate into a single struct. The first observation for any counter always yields delta 0.

Two CQ data paths. An important subtlety: CQ metrics arrive via two independent paths, and they populate *different* fields:

- **BPF map snapshot path** (`ingest_bpf_snapshot`): CQ data from `CQ_JITTER_STATS` writes to `cq_jitter` (the field used by live detection).
- **Ring buffer event path** (`ingest` with `CqCompletion`): writes to `rdma_metrics` and the `DDSSketch`. This path is active in mock/replay mode.

The `RdmaLatencySpikeRule` reads `rdma_metrics` (event path), while `CqJitterRule` reads `cq_jitter` (BPF map path). On live systems, only the BPF map path is active, so `RdmaLatencySpike` is inert.

3.3 Window timing and its implications

```
let window_duration = Duration::from_secs(config.window_secs); // default: 3s
let tick_interval   = Duration::from_millis(200);              // poll interval
let mut window_start = Instant::now();
// On each iteration:
if window_start.elapsed() >= window_duration {
    // ... collect, evaluate, export, reset ...
    window_start = Instant::now();
}
thread::sleep(tick_interval); // 200ms between checks
```

Listing 2: Window timing from `main.rs`

What this means in practice.

- The agent checks whether the window has elapsed every 200 ms. This means actual window length is 3.0–3.2 s ($\pm$ one tick).
- All data collection, rule evaluation, smoothing, state machine evaluation, exporter updates, and scheduler reconciliation happen synchronously on the same thread during a single window boundary.
- Before the main loop begins, `main.rs` performs one warmup BPF map read. This primes the previous-value buffers so that the first real window computes meaningful deltas (not “everything since boot”).

Caution

The sysfs counter `HashMap` starts empty. The first sysfs read yields `delta = 0` for all counters (no previous value to subtract from). IB counter deltas become meaningful starting from window 2.

4 Detection Logic

This section explains how the engine turns a window of aggregated metrics into a health score. The process has four stages, each described below: composite scoring, carry-forward, capability fusion, and coverage weighting.

4.1 Stage 1: Composite raw score

The idea. The engine weighs multiple independent signals and sums them into a single score in $[0, 1]$. A score of 0 means “no problems detected.” A score approaching 1 means “severe multi-signal

degradation.”

Different signals contribute different maximum amounts. This reflects their diagnostic importance. For example, a downed link is far more significant than elevated slab allocation latency.

The formula. The composite score is a weighted sum of nine terms, computed by `compute_health_score` in `detection/mod.rs`:

Signal	Max weight	How it contributes
Link error recovery (> 0)	0.35	Binary: any recovery attempt in this window adds 0.35. This is the strongest single signal—it precedes link failure.
Link downed (> 0)	0.25	Binary: a downed link adds 0.25.
Operational/soft errors	0.25	Proportional to <code>soft_errors/throughput</code> $\times 100$, clamped at 1.0 before weighting. Only counted when traffic is present. Covers RXE duplicate-request, sequence-error, retry-exceeded, and send-error counters. On real IB, these are zero and the hard-error term dominates instead.
Hard IB errors	0.15	Proportional to <code>hard_errors/throughput</code> $\times 1000$, clamped at 1.0 before weighting.
IRQ skew	0.15	Proportional to how far the dominant CPU’s interrupt share exceeds a baseline ($100/n_{\text{cpu}} + 20$).
CQ stall ratio	0.15	Ratio of stalled completions ($> 50 \mu\text{s}$) to total completions.
Congestion spread	0.10	Binary: <code>port_xmit_wait</code> > 0 and no local hard errors. Detects credit stalls from upstream congestion.
NAPI saturation	0.05	Activates when average work/budget > 0.7 .
Slab pressure	0.05	Activates when average slab alloc latency $> 1 \mu\text{s}$.

Table 2: Composite score terms, from `compute_health_score`.

The result is clamped: $\text{score} = \min(\text{sum of all terms}, 1.0)$.

Why these weights? The weights reflect a priority hierarchy. Link-level signals (recovery, downed, hard errors) have the highest combined weight because they indicate physical or firmware faults that require human intervention. Soft soft errors carry meaningful weight because on software-RDMA devices (RXE) they are the primary observable for packet loss; on real IB fabrics the hard-error and link-recovery terms dominate instead. Host-level signals (IRQ, slab, NAPI) have smaller weights because they are correlating indicators, not root causes.

Key Insight

If a link goes down (recovery + downed), the score immediately reaches at least $0.35 + 0.25 = 0.60$, which exceeds the `critical_enter` = 0.55 threshold. A single window of link failure can begin the path to Critical.

4.2 Stage 2: IB-aware carry-forward

The problem. When packet loss occurs on an RDMA link, the sender’s queue pair enters retransmission backoff. During backoff, there is no traffic, so error counters stop incrementing. If the engine only looked at fresh metrics, it would see “no errors” and immediately start recovering—even though the impairment is still present.

The solution. When the engine detects *no IB signal* (no traffic, no errors, no link events) in a window but had a nonzero score previously, it carries forward the previous raw score with a 10% per-window decay:

```
let has_ib_signal = d.has_traffic()
    || d.total_soft_error_delta() > 0
    || d.total_hard_error_delta() > 0
    || d.link_error_recovery_delta > 0
    || d.link_downed_delta > 0;

if !has_ib_signal {
    // No RDMA activity at all. Carry previous score at 0.9x decay.
    let carry = prev_raw_score * 0.9;
    return carry.max(fresh);    // But if host signals push higher, let them.
}
return fresh;
```

Listing 3: Carry-forward logic (detection/mod.rs)

The math. Starting from a raw score of r_0 with no subsequent IB signal, the carried score after k windows is $r_0 \cdot 0.9^k$. To decay from 0.6 to below 0.3:

$$k = \left\lceil \frac{\log(0.3/0.6)}{\log(0.9)} \right\rceil = 7 \text{ windows} \approx 21 \text{ s}$$

Caution

If an operator stops the RDMA workload as a remediation (no traffic, no errors), the carry-forward extends recovery by ~ 21 seconds. This is a deliberate trade-off: it is better to hold a degraded verdict during backoff than to flap between healthy and critical.

4.3 Stage 3: Capability fusion

The idea. In addition to the rule-derived score (Stage 1), the engine can receive supplementary signals from *capability providers*—backends that read retransmission counters, link error rates, peer liveness probes, and other data. These signals are fused into the score as an additive boost.

Why additive-only? The fusion contribution is always non-negative. A capability provider reporting “all is well” does *not* reduce the score. This ensures that the engine never becomes *less* conservative because of additional data. The rationale from the code: “*the rule layer can still drive the score all the way to critical, enter on its own, so the capability contribution alone is never enough to push state into Critical without rule corroboration.*”

The computation. For each Sample s with capability c :

1. **Normalize:** map the raw value to $[0, 1]$ using a capability-specific function $n(s)$. (Example: for CompletionLatency, $n = (v - 100 \mu s) / 900 \mu s$.)
2. **Weight:** multiply by the capability’s importance weight w_c (range: 0.0 for metadata-only capabilities to 1.0 for LinkErrors).
3. **Quality-scale:** multiply by the sample’s effective weight (`quality.weight()` $\times$ `confidence`). An Absent-quality sample contributes zero.

The sum of all contributions is then squashed:

$$\Delta_{\text{fusion}} = \text{clamp}\left(\tanh\left(\frac{S}{2}\right) \cdot 0.5, 0, 0.5\right) \quad \text{where} \quad S = \sum_s n(s) \cdot w_{c(s)} \cdot s.\text{effective_weight}()$$

The tanh squash ensures the contribution is bounded at 0.5 regardless of how many signals fire. This means *capability signals alone can never push the score past 0.5*—rule corroboration is required to reach `critical_enter` (0.55).

4.4 Stage 4: Coverage-weighted severity floor

The problem. A detection rule might fire (e.g., “link errors above budget”) based on partial data, but the underlying signals it depends on might be unavailable on this hardware. If the engine blindly trusts the rule, it might drain a node based on data it cannot verify.

The solution. Each rule declares which capabilities it consults. When a rule fires, the engine checks the `CoverageReport` to see how good the data is for those capabilities. The severity floor—the minimum raw score that a rule verdict can impose—is scaled by this quality:

$$\text{floor} = \underbrace{\begin{cases} 0.55 & \text{worst rule severity} = \text{Critical} \\ 0.30 & \text{worst rule severity} \in \{\text{Degraded}, \text{Recovering}\} \\ 0 & \text{worst rule severity} = \text{Healthy} \end{cases}}_{\text{floor}_{\text{base}}} \times \underbrace{\max_{c \in C} \text{quality}(c).\text{weight}()}_{\text{coverage weight } w_{\text{cov}}}$$

What this means in practice.

- If all consulted capabilities are **High** quality: $w_{\text{cov}} = 1.0$. The floor applies at full strength.
- If all consulted capabilities are **Absent**: $w_{\text{cov}} = 0.0$. The floor is zero—the rule cannot push the score upward on its own.
- If at least one consulted capability is **High**: $w_{\text{cov}} = 1.0$ (the *max* operator means the best evidence wins).
- Rules that consult no capabilities (host-signal rules like `NapiSaturation`) always get $w_{\text{cov}} = 1.0$.

4.5 Putting it all together: one evaluation cycle

The following pseudocode shows the complete per-window computation, with annotations explaining each step:

```
fn evaluate_with_coverage(metrics, samples, coverage):
    // --- Step 1: Run all 11 rules ---
    // Each rule examines the aggregated metrics and optionally emits an alert.
    // We track the worst severity seen, and which capabilities that rule
    // depends on (for the coverage floor later).
    alerts = []
    worst_severity = Healthy
    worst_caps = []
    for rule in rules:
        if let Some(alert) = rule.evaluate_mut(metrics):
            if severity_rank(alert.severity) > severity_rank(worst_severity):
                worst_severity = alert.severity
```

```

        worst_caps      = rule.capabilities_consulted()
        alerts.push(alert)

// --- Step 2: Compute the raw score ---
// Weighted sum of 9 signal terms. If no IB signal exists, carry
// forward previous score at 0.9x decay.
raw = compute_effective_raw(metrics)

// --- Step 3: Add capability fusion contribution (bounded at 0.5) ---
raw = min(raw + sample_score_contribution(samples), 1.0)

// --- Step 4: Apply coverage-weighted severity floor ---
// Ensures the raw score is at least as high as the floor demands,
// scaled by how much we trust the data behind the worst-firing rule.
cov_w = coverage.input_weight(worst_caps) // max(quality.weight())
floor = severity_floor_for(worst_severity) * cov_w
raw    = max(raw, floor)

// --- Step 5: Smooth the raw score ---
// EWMA absorbs noise; peak-hold remembers recent spikes.
effective = smoothed_score.update(raw)

// --- Step 6: Feed the smoothed score to the state machine ---
// The state machine applies dwell timers and hysteresis.
state_machine.evaluate(effective)

// --- Step 7: Parallel analysis (telemetry only) ---
multi_timescale.evaluate(raw)           // fast/medium/slow tracks
burst_classifier.observe(raw, &multi_timescale)

return alerts

```

Listing 4: Complete evaluation pseudocode (detection/mod.rs)

5 Smoothing: From Raw Score to Effective Score

5.1 Why smooth at all?

The raw score can swing wildly between windows. Consider 20% packet loss on RDMA traffic under packet loss: errors spike, the sender enters retransmission backoff, errors disappear, then traffic resumes and errors spike again. The raw score alternates between ~ 0.6 and ~ 0.0 on a per-window basis. If the state machine saw these swings directly, it would flap between Healthy and Critical every few seconds.

Smoothing converts this into a stable trend that the state machine can act on reliably.

5.2 The dual-track smoother

`SmoothedHealthScore` (detection/mod.rs) combines two tracks and takes their maximum:

Track 1: EWMA (Exponentially Weighted Moving Average). This is a standard low-pass filter. With smoothing factor α , each new raw value blends into the running average:

$$\text{ewma}_t = \text{ewma}_{t-1} + \alpha \cdot (\text{raw}_t - \text{ewma}_{t-1})$$

A higher α makes the EWMA more responsive. Default: $\alpha = 0.3$.

On its own, the EWMA is not sufficient. If the raw score alternates between 0.6 and 0.0, the EWMA converges to ~ 0.3 —near the degrade threshold, but not above the critical threshold. The system would sit in Degraded when it should be in Critical.

Track 2: Peak-hold with geometric decay. This track remembers the highest recent score and forgets it slowly:

$$\text{peak}_t = \begin{cases} \text{raw}_t & \text{if } \text{raw}_t \geq \text{peak}_{t-1} \\ \text{peak}_{t-1} \cdot \rho & \text{otherwise (geometric decay)} \end{cases}$$

Default decay: $\rho = 0.85$. A spike of 0.6 decays as:

Window after spike	0	1	2	3	4	5	6	7	8
Peak-hold value	0.600	0.510	0.434	0.368	0.313	0.266	0.226	0.192	0.163

After 5 clean windows, the peak has decayed to 0.266—below `critical_exit` (0.30). After 11 clean windows, it falls below `degrade_exit` (0.10).

The effective score.

$$\text{effective}_t = \max(\text{ewma}_t, \text{peak}_t)$$

Key Insight

The peak-hold track is the single most important anti-flap mechanism. It ensures that a single bad window ($\text{raw} = 0.6$) keeps the effective score above `critical_exit` for at least 5 more windows, even if every subsequent window is perfectly clean. This prevents the state machine from prematurely de-escalating.

6 The State Machine

6.1 Why a state machine?

Even after smoothing, the effective score can briefly dip below or above thresholds. If the engine changed state on every threshold crossing, it would still flap. The state machine adds **dwelling requirements**: a condition must hold for multiple consecutive windows before a transition occurs.

Additionally, escalation (getting worse) is fast while de-escalation (recovering) is slow. This asymmetry reflects a core operational principle: *it is far worse to return a faulty node to service than to keep a healthy node out of service for an extra minute.*

6.2 The four states

Healthy The node has no sustained problems. Workloads can be scheduled normally.

Degraded The effective score has been above `degrade_enter` (0.30) for at least `enter_windows` (2) consecutive windows. Something is wrong but not catastrophic. The scheduler may begin draining.

Critical Escalated from Degraded. The effective score has been above `critical_enter` (0.55) for at least 2 more windows. The node has a serious problem. The scheduler should drain immediately.

Recovering

The node was Critical but conditions have improved. This is a mandatory holding state—the engine waits here for `recover_windows` (3) windows before moving to Degraded. This prevents returning to service too quickly if the fault reappears. For scheduler purposes, **Recovering** maps to **Degraded** (the node stays drained).

6.3 Configuration defaults

Parameter	Default	Meaning
<code>degrade_enter</code>	0.30	Effective score to begin escalation from Healthy.
<code>degrade_exit</code>	0.10	Effective score to begin de-escalation from Degraded.
<code>critical_enter</code>	0.55	Effective score to begin escalation from Degraded.
<code>critical_exit</code>	0.30	Effective score to begin de-escalation from Critical.
<code>enter_windows</code>	2	Consecutive windows above threshold to escalate.
<code>exit_windows</code>	5	Consecutive windows below threshold to de-escalate.
<code>recover_windows</code>	3	Mandatory hold in Recovering before returning to Degraded.
<code>max_hold_windows</code>	200	Watchdog: log a warning if stuck in any non-Healthy state.

Key Insight

Entry requires 2 consecutive windows; exit requires 5. This 2.5x asymmetry means the engine escalates $\sim 2.5\times$ faster than it de-escalates. Combined with the peak-hold smoother, a 6-second fault takes ~ 54 seconds of clean input to fully recover from.

6.4 Transition rules

Each transition has an evidence counter that increments when the condition is met and resets to zero when it is not. A transition fires when the counter reaches the required dwell.

Healthy $\rightarrow$ Degraded.

- Each window: if effective ≥ 0.30 , increment `escalation_evidence`. Otherwise, reset it to 0.
- Transition: when `escalation_evidence` ≥ 2 .

Degraded $\rightarrow$ Critical.

- Each window: if effective ≥ 0.55 , increment `escalation_evidence`. Otherwise, reset it.

- Transition: when `escalation_evidence` ≥ 2 .
- De-escalation check (only if escalation did not fire this window): if `effective` < 0.10 , increment `stability_evidence`. Otherwise, reset it. On ≥ 5 , transition to Healthy.

Caution

In the Degraded state, a window where $0.10 \leq \text{effective} < 0.55$ makes progress toward *neither* direction. Both evidence counters are reset. The node remains Degraded until the effective score moves decisively.

Critical $\rightarrow$ Recovering.

- Each window: if `effective` < 0.30 , increment `stability_evidence`. Otherwise, reset it.
- Transition: when `stability_evidence` ≥ 5 .

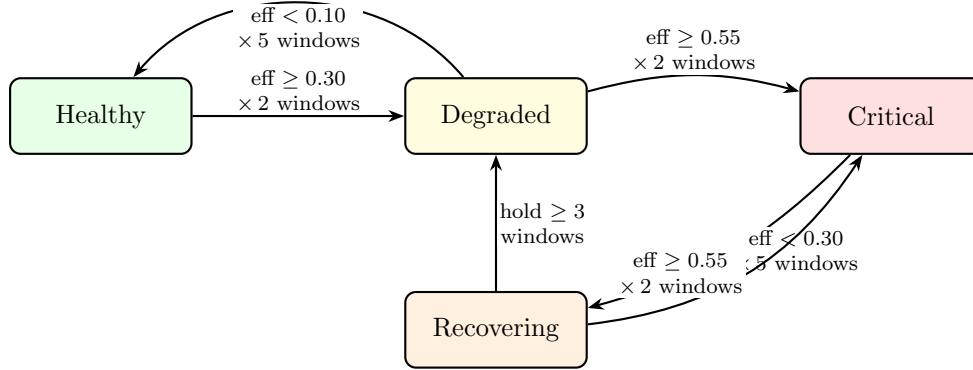
There is no upward escalation from Critical. It is the worst state.

Recovering $\rightarrow$ Degraded or Critical.

- Each window: increment `hold_counter`. When ≥ 3 , transition to Degraded.
- Re-escalation: if `effective` ≥ 0.55 for ≥ 2 consecutive windows, transition back to Critical.

On every transition. All evidence counters (`escalation_evidence`, `stability_evidence`, `hold_counter`, `windows_in_state`) are reset to zero.

6.5 Transition diagram



There is no direct path from Critical to Healthy or from Critical to Degraded. Recovery *always* passes through Recovering.

6.6 Worked example: 20% packet loss, then recovery

This walkthrough traces the engine's behavior window-by-window. Assume 3s windows, default config, and an InfiniBand device with active RDMA traffic.

Win	Event	Raw	Peak	Effective	State
1	Cable fault begins. Symbol errors spike.	0.58	0.58	0.58	Healthy (esc=1)
2	Errors continue, link_error_recovery fires.	0.60	0.60	0.60	Degraded (esc=1)
3	RDMA backoff: no traffic.	0.54	0.54	0.54	Degraded (esc=0)
4	Traffic resumes with errors.	0.58	0.58	0.58	Critical (esc=2)
5	Operator replaces cable. Clean traffic.	0.00	0.49	0.49	Critical (stab=0)
6	Clean traffic.	0.00	0.42	0.42	Critical (stab=0)
7	Clean traffic.	0.00	0.36	0.36	Critical (stab=0)
8	Clean traffic.	0.00	0.30	0.30	Critical (stab=0)
9	Clean traffic.	0.00	0.26	0.26	Critical (stab=1)
10	Clean traffic.	0.00	0.22	0.22	Critical (stab=2)
11	Clean traffic.	0.00	0.18	0.18	Critical (stab=3)
12	Clean traffic.	0.00	0.16	0.16	Critical (stab=4)
13	Clean traffic.	0.00	0.13	0.13	Recovering (hold=1)
14	Clean traffic.	0.00	0.11	0.11	Recovering (hold=2)
15	Clean traffic.	0.00	0.10	0.10	Degraded (hold $\geq$ 3)
16–20	Clean traffic. Effective decays below 0.10.	0.00	<0.10	<0.10	Degraded (stab 1..5)
21	Five consecutive < 0.10.	0.00	0.05	0.05	Healthy

Key observations.

- Escalation to Critical took ~ 4 windows (~ 12 s).
- Recovery from Critical to Healthy took ~ 17 windows (~ 51 s) of perfectly clean input.
- The peak-hold track is what kept the effective score above 0.30 for windows 5–8 even though raw was 0.00. Without it, the engine would have exited Critical after just 5 clean windows.

7 Detection Rules

7.1 Overview

The detection engine runs 11 rules. Each examines the per-window **AggregatedMetrics** and optionally returns an **Alert** with a severity (**Degraded** or **Critical**).

Rules serve two purposes:

1. They produce human-readable alerts (“what is wrong?”).
2. Their worst severity drives the coverage-weighted floor in the composite score (“how bad is it?”).

Rules that depend on fabric signals declare which capabilities they consult. Rules operating only on host-level signals (IRQ, NAPI, slab) consult no capabilities and always receive full coverage weight.

7.2 Rule reference

7.3 Rule detail: InterruptAffinitySkew

Detects uneven IRQ distribution across CPUs. On modern systems with RSS (Receive Side Scaling) and **irqbalance**, interrupts should be spread across cores. A single CPU handling a disproportionate share indicates a misconfiguration or a saturated interrupt coalescing path.

Guard conditions. The rule is silenced when interrupt volume is too low to be meaningful: < 10 total IRQs per window (general), or < 1000 on systems with ≤ 2 CPUs (where imbalance is structurally unavoidable).

Rule name	Caps	When it fires and at what severity
InterruptAffinitySkew	—	Dominant CPU pct $\geq \max(70, 100/n + 45)$. Degraded; Critical if $\geq 90\%$. Needs ≥ 10 IRQs (≥ 1000 on ≤ 2 -CPU).
RdmaLatencySpike	CL	Avg CQ latency $\geq 5\times$ baseline ($2\mu\text{s}$). Needs ≥ 10 completions. Critical at $\geq 10\times$. <i>Only fires in mock/replay mode.</i>
RdmaLinkDegradation	LE, RS	<i>Stateful, 5-layer.</i> See Section 7.9 for detailed explanation.
SlabPressure	—	≥ 5000 slab allocs <i>and</i> IB errors > 0 . Correlation detector. Critical at > 100 errors.
RisingErrorTrend	LE	Total errors monotonically increasing for ≥ 3 windows. Critical at ≥ 6 .
LatencyDrift	CL	Slab alloc latency z-score ≥ 3 vs EWMA baseline. Always Degraded.
ThroughputDrop	TH	IB throughput drops $\geq 50\%$ below EWMA baseline (floor-clamped at $0.5\times$ initial). Critical at $\geq 80\%$.
NapiSaturation	—	Avg NAPI work/budget $\geq 85\%$. Critical at $\geq 95\%$.
CqJitter	CL	Any stall ($> 50\mu\text{s}$ completion). Immediate alert. Critical if > 10 stalls or max $> 1\text{ms}$. Also: p99 anomaly via z-score.
CongestionSpread	PP, CS	<code>xmit_wait</code> z-score ≥ 3 <i>and</i> no local hard errors. Detects upstream congestion (“victim node”). Critical at $z \geq 6$.
PcieBottleneck	—	CQ stalls ≥ 5 <i>and</i> slab ≥ 1000 <i>and</i> no IB errors. Always Degraded.

Table 3: Detection rules. Capabilities: CL = CompletionLatency, LE = LinkErrors, RS = RetransmitSignal, TH = Throughput, PP = PfcPause, CS = CreditStall.

Effective threshold. The threshold adapts to CPU count: $\text{effective} = \max(\text{threshold_pct}, 100/n_{\text{cpus}} + 45)$. On a 2-CPU system this is $\max(70, 95) = 95\%$, making the rule near-impossible to fire (by design—imbalance is expected with 2 CPUs).

7.4 Rule detail: LatencyDrift

Detects anomalous slab allocation latency relative to the node’s own baseline. Uses a **RollingStats** EWMA tracker ($\alpha = 0.1$) to establish what is “normal” for this specific system, then fires when the current window’s average deviates by ≥ 3 standard deviations.

Guard conditions. Requires slab latency $\geq 1000\text{ns}$ (filters out the common case of sub-microsecond allocations on idle systems). Also requires `has_network_signal`—at least one of: IB hard errors, IB soft errors, `rdma_metrics.error_count > 0`, or `cq_jitter.stall_count > 0`. This makes it a **correlation detector**: slab latency anomalies alone do not trigger alerts; they must co-occur with network-layer indicators.

Severity. Always Degraded (never Critical). This reflects that slab latency is a correlating signal, not a root cause.

7.5 Rule detail: ThroughputDrop

Monitors IB throughput (bytes or packets, whichever is available) via a **RollingStats** tracker with a floor clamp. The tracker uses $\alpha = 0.1$ and prevents the EWMA from dropping below $0.5\times$ the initial baseline. This means a legitimate workload reduction eventually stops triggering, but a sudden drop always fires.

Thresholds.

- $\geq 50\%$ drop from EWMA baseline: **Degraded**
- $\geq 80\%$ drop: **Critical**

7.6 Rule detail: CqJitter

Two independent detection paths:

1. **Stall detector**: If `cq-jitter.stall_count` > 0 (any completion $> 50\mu\text{s}$), fire immediately. Escalate to **Critical** if > 10 stalls or max latency $> 1\text{ms}$.
2. **p99 anomaly**: Uses a `RollingStats` tracker with clamp $[0.1, 5.0]$ on the estimated p99 latency. After warmup, fires **Degraded** if z-score ≥ 3 .

7.7 Rule detail: CongestionSpread

Detects upstream fabric congestion affecting this node as a “victim.” The key insight: when a remote node or switch is congested, IB flow control causes `port_xmit_wait` to rise on the local port—but the local node has no hard errors of its own.

Guard conditions. Requires: (i) warmup complete, (ii) `xmit_wait` ≥ 1 , (iii) `total_hard_error_delta + link_error_recovery_delta` $= 0$ (no local errors), (iv) z-score ≥ 3 .

Why exclude local errors. If the node has its own hard errors, the signal is likely a local problem, not upstream congestion. The rule only fires when the node itself is clean but being starved by its neighbors.

7.8 Rule detail: PcieBottleneck

A heuristic detector for PCIe bus saturation. Fires when CQ stalls (≥ 5) and slab pressure (≥ 1000 allocs) co-occur *without* any IB errors. The logic: if completions are stalling and memory allocation is pressured, but the IB link is healthy, the bottleneck is likely between the NIC and CPU (PCIe).

Always **Degraded**—never **Critical**—because this is an indirect inference, not a direct observation.

7.9 Deep dive: RdmaLinkDegradation (the most complex rule)

This rule models the real-world failure sequence for an InfiniBand cable: *Nominal* $\rightarrow$ *Elevated Errors* $\rightarrow$ *Link Recovery* $\rightarrow$ *Link Down*. It is stateful, maintains three `RollingStats` trackers (symbol errors, receive errors, soft errors), an `elevated_windows` counter (capped at 10), and a cooldown timer. It uses five detection layers, checked in priority order from most severe to least:

1. **Link down** (`link_downed_delta` > 0): Immediately **Critical**. The cable is dead; replace it.
2. **Link recovery** (`link_error_recovery_delta` > 0): Immediately **Critical**. The firmware is actively trying to recover a failing link. This event precedes link-down and is the earliest warning of a dying cable.
3. **Error budget exceeded** (`total_hard_error_delta` $>$ `error_budget`, default 5 per window): **Degraded**, or **Critical** if $> 10\times$ budget (50). The budget is a per-window count, not a rate—this fires even at low throughput.

4. **Absolute error rate ceiling** ($\geq 5\%$): On links with traffic, if `total_all_errors_delta()/throughput` ≥ 0.05 fires unconditionally. This catches sustained degradation that the EWMA might adapt to (the “boiling frog” scenario). On Soft-RoCE devices, uses soft-error totals instead.
5. **Rate anomaly via z-score** ($z \geq 3$ after warmup): Computes $z = \max(z_{\text{sym}}, z_{\text{rcv}}, z_{\text{soft}})$ using clamped EWMA trackers. Requires `total_errors` > 0 . Critical at $z \geq 2 \times \text{threshold}$ (6), or after ≥ 3 consecutive elevated windows, or if the total error rate $\geq 3 \times$ the absolute ceiling (15%).

State management. When any layer fires, `elevated_windows` increments (capped at 10) and `cooldown_remaining` resets to 3. When no layer fires and cooldown is active, `cooldown_remaining` decrements. When cooldown expires, `elevated_windows` decrements (step-down). This provides a memory of recent degradation that decays gradually.

The “boiling frog” defense. Rules that use EWMA baselines have a risk: if an error rate rises slowly enough, the EWMA adapts to it and the z-score stays low. The `RollingStats` tracker addresses this with *clamping*: after a 5-window warmup, it captures the EWMA as a baseline and prevents the EWMA from drifting above $3 \times$ baseline. This means a slowly rising error rate will eventually exceed the clamped EWMA and trigger an anomaly.

The absolute error rate ceiling (layer 4) provides a second defense: regardless of what the EWMA thinks is “normal,” any window with $\geq 5\%$ error rate fires unconditionally.

7.10 RollingStats: the statistical backbone

Several rules (`RdmaLinkDegradation`, `LatencyDrift`, `ThroughputDrop`, `CqJitter`, `CongestionSpread`) depend on the `RollingStats` tracker (`detection/rolling_stats.rs`). Its behavior is central to understanding why rules fire when they do.

EWMA update. On each observation x_t :

$$\text{ewma}_t = \text{ewma}_{t-1} + \alpha \cdot (x_t - \text{ewma}_{t-1})$$

After update, the EWMA is clamped: $\text{ewma}_t \leftarrow \min(\text{ewma}_t, \text{baseline} \times \text{clamp_hi})$, where the baseline is frozen after warmup. Without this clamp, a gradually worsening signal would drag the EWMA upward and the z-score would never fire.

Variance tracking. Uses an EWMA of squared deviations:

$$\text{var}_t = \text{var}_{t-1} + \alpha \cdot ((x_t - \text{ewma}_t)^2 - \text{var}_{t-1})$$

Standard deviation: $\sigma_t = \sqrt{\text{var}_t}$.

Z-score.

$$z_t = \frac{x_t - \text{ewma}_t}{\max(\sigma_t, \epsilon)}$$

where ϵ is a small floor to prevent division by near-zero. A $z \geq 3$ indicates the observation is 3+ standard deviations above the expected value.

Floor clamp. `RollingStats::with_floor( $\alpha$ , floor_factor)` prevents the EWMA from dropping below `floor_factor` $\times$ initial. Used by `ThroughputDrop` with floor 0.5: if throughput drops to 30% of baseline, the EWMA stops at 50%, making the z-score reflect the full gap. Without the floor, a gradual workload ramp-down would drag the EWMA down and mask a genuine drop.

TrendTracker. A companion type that tracks *monotonically increasing* sequences. Each window, if $x_t > x_{t-1}$, `consecutive_increases` increments; otherwise it resets. Used by `RisingErrorTrendRule` to detect sustained degradation trajectories.

8 Multi-Timescale Evaluation and Burst Classification

8.1 Why multiple timescales?

A single smoother with $\alpha = 0.3$ responds in a few windows. This is good for detecting sudden faults but blind to slow, multi-minute drift. Conversely, a slow smoother ($\alpha = 0.1$) catches drift but ignores brief bursts.

The engine solves this by running three parallel `SmoothedHealthScore` + `HealthStateMachine` pairs, each with different parameters:

Track	EWMA α	Peak decay ρ	Enter dwell	Exit dwell
Fast	0.6	0.5	1	2–3
Medium	0.3	0.85	2	5
Slow	0.1	0.95	4	8–10

The composite verdict across the three tracks is the **worst observed state**.

Caution

Important caveat: the multi-timescale evaluator is currently published for telemetry and consumed by the burst classifier, but it does **not** drive the authoritative health state. The single-track Medium state machine alone governs `current_state()` and the scheduler. This means operators can see `Critical` on the fast track in Grafana while the official state is `Degraded`.

8.2 Burst classification

The `BurstClassifier` (`detection/burst.rs`) uses a sliding window of 30 recent evaluations. Any window where the raw score ≥ 0.40 counts as a “burst event.” The classifier then cross-references the timescale states:

Classification	Condition	Meaning
Quiet	No bursts, slow track Healthy	Normal operation.
Burst	Bursts detected, slow track Healthy	Short spikes—probably transient.
Sustained	No bursts, slow track Degraded/Critical	Slow drift without sharp peaks.
MixedBurstSustained	Bursts + slow track Degraded/Critical	Both burst and drift occurring.

This classification is exposed via the `argus.burst_classification` Prometheus gauge and is intended for use by the scheduler to differentiate responses (e.g., a burst warrants caution but not immediate drain).

9 Timing Model

9.1 Time constants at a glance

With the default 3s window, the real-world time for each transition:

Transition	Min windows	Wall-clock time
Healthy $\rightarrow$ Degraded	2	~ 6 s
Degraded $\rightarrow$ Critical	2	~ 6 s
Critical $\rightarrow$ Recovering	5 + peak-hold decay	~ 27 s
Recovering $\rightarrow$ Degraded	3	~ 9 s
Degraded $\rightarrow$ Healthy	5	~ 15 s
Healthy $\rightarrow$ Critical (min)	4	~ 12 s
Critical $\rightarrow$ Healthy (min)	~ 18	~ 54 s

Note on peak-hold. The “Critical $\rightarrow$ Recovering” transition requires not just 5 windows of effective < 0.30 , but 5 consecutive windows where the peak-hold track has decayed below 0.30. From a peak of 0.6, this decay alone takes 5 windows ($0.6 \times 0.85^5 = 0.27$). Combining peak-hold decay and dwell requirements, at least ~ 9 windows elapse before reaching Recovering.

9.2 Timing edge cases

Window boundary jitter. The tick loop sleeps 200 ms between checks. Actual window duration varies between $[3.0, 3.2]$ s. Over many windows, this jitter is inconsequential—but it means transitions can take one tick longer than the minimum.

Capability provider latency. Providers run sequentially within `collect_all`. There is no per-provider timeout. A slow sysfs read extends the evaluation phase, potentially skewing the next window boundary.

10 Failure Modes

This section documents known limitations and surprising behaviors, each tied to specific implementation details.

10.1 Recovery takes longer than expected

Root cause. The peak-hold decay, the 5-window dwell for exiting Critical, the 3-window hold in Recovering, and another 5-window dwell for exiting Degraded combine to require ~ 18 windows (~ 54 s) of clean input to return to Healthy from Critical. This is intentional—verified by the test `full_recovery_path_critical_to_healthy`.

Operator impact. An operator who removes a fault (e.g., by deleting a `tc netem` rule) will see the node remain in Critical/Recovering/Degraded for roughly one minute.

10.2 Carry-forward extends recovery when workload stops

Root cause. If the RDMA workload stops entirely (no traffic, no errors), the carry-forward mechanism (Section 4.2) keeps the raw score elevated at 0.9^k of its previous value. From 0.6, this takes 7 windows (~ 21 s) to decay below 0.30.

Operator impact. Draining the workload as a remediation does not immediately signal recovery.

10.3 Missing or incomplete sysfs counter layout

Root cause. Not all RDMA devices expose the same sysfs counters. On real IB HCAs, the standard `counters/` directory provides byte-level throughput (`port_rcv_data`, `port_xmit_data`) and all hard-error counters—the engine operates at full fidelity. On software RDMA devices (RXE/Soft-RoCE) that expose only `hw_counters/`, `throughput_bytes()` returns 0 and the engine falls back to `throughput_pkts()`. If neither byte nor packet counters are available, `has_traffic()` returns false and the soft-error score term contributes nothing.

Operator impact. On devices with incomplete counter layouts, the engine is under-sensitive to RDMA-specific faults. The score is driven only by IRQ, slab, NAPI, and CQ jitter signals. This primarily affects Soft-RoCE test environments; real IB deployments are unaffected.

10.4 Capability fusion never reduces the score

Root cause. The fusion contribution is ≥ 0 by construction. A capability provider that reports “no problems” contributes $\Delta = 0$, not a negative value.

Operator impact. If a rule fires based on partial data and the capability providers have better data showing the node is healthy, the providers cannot override the rule. The only attenuation path is the coverage-weighted floor (Section 4.4), which requires the consulted capabilities to be Absent.

10.5 Determinism with caveats

The engine itself is deterministic: the test `state_machine_is_deterministic` verifies that the same raw-score sequence always produces the same state sequence. However, inputs are non-deterministic: sysfs reads and BPF map reads happen on the agent thread without synchronization with the kernel, and 200 ms tick jitter determines which observations land in which window.

11 Observability and Debuggability

11.1 What is exported

The agent exposes the following via Prometheus (`/metrics`) and JSON endpoints:

Health state and score. `argus_health_state` (0/1/2/3 for H/D/C/R), `argus_health_score_millis`, `argus_health_score_raw_millis`, `argus_health_score_ewma_millis`, `argus_health_score_peak_hold_millis`, `argus_health_score_effective_millis`.

Capability layer. `argus_capability_coverage`{`capability`,`quality`}, `argus_capability_grade`, `argus_sample_contribution_millis`.

Multi-timescale. `argus_timescale_state`{`timescale`}, `argus_timescale_score_millis`{`timescale`}.

Alerts and transitions. `argus_alert_count`{`kind`,`severity`}, `argus_state_transitions`{`from`,`to`}.

HTTP endpoints. `/metrics` (Prometheus), `/health` (state JSON), `/status` (full JSON with recent alerts), `/coverage` (capability coverage report).

Structured logs. `tracing::info!` on every state transition (with `from`, `to`, `seq`) and on capability tier selection at startup. `tracing::warn!` on the stuck-state watchdog (200 windows in one state).

11.2 What is not exported

- **Per-rule firing rate.** There is no “rule X fired 8 of the last 10 windows” metric.
- **Carry-forward indicator.** An operator cannot tell from exported data whether a high score comes from fresh signal or carry-forward. The `prev_raw_score` field is internal.
- **Rule evidence.** `RuleEvidence` is defined in `fusion.rs` but not emitted on the `Alert` struct.
- **Link degradation rule internals.** The `elevated_windows`, `cooldown_remaining`, and `link_state` fields of `RdmaLinkDegradationRule` are not surfaced.

11.3 How to trace a decision

When a node is in an unexpected state, follow these steps:

1. **Read `/status`.** Contains the full `AggregatedMetrics` and the last 100 alerts.
2. **Compare raw vs. peak hold.** If `peak_hold_millis` $\gg$ `raw_millis`, the engine is in residual hold from a past spike. The node will stay in its current state until peak decays below the exit threshold.
3. **Check `/coverage`.** If a rule’s consulted capability is `Absent`, the coverage-weighted floor for that rule is 0; the rule’s verdict alone cannot push the score up.
4. **Compare timescale states.** If the fast track says `Critical` and the slow track says `Healthy`, the problem is a short burst. If the slow track also says `Critical`, it is a sustained problem.
5. **Check `state_transitions`{`from`,`to`}.** Persistent flap (many transitions) is evidence of an oscillating *input*, not an engine bug. The default config makes flap from clean windows impossible (verified by tests).

12 Limitations and Gaps

12.1 Signals the engine cannot observe today

CQ latency via eBPF.

`RdmaLatencySpikeRule` reads `rdma_metrics`, populated only in mock/replay mode. Live eBPF kprobes currently attach to RXE-specific symbols (`rx_post_send/rx_poll_cq`). On production IB hardware (`mlx5`, `hfi1`), these symbols do not exist and the rule is inert. Vendor-specific kprobe targets for `mlx5` CQ completion paths have not yet been added.

PCIe link-state changes.

`PcieBottleneckRule` infers PCIe pressure indirectly (CQ stalls + slab pressure – IB errors). It cannot observe PCIe AER events.

Counter wraparound.

32-bit IB counters are handled via `saturating_sub`. A wrap produces `delta = 0` for one window (under-detection), then normal deltas resume.

Asymmetric faults.

The engine has no concept of “the remote end caused this.” A receiver node experiencing degradation from a bad remote sender may not produce local signals. Cross-node correlation is external.

12.2 Assumption failures

Devices with incomplete sysfs counters.

On devices (primarily Soft-RoCE/RXE) where neither byte nor packet throughput counters are available, `has_traffic()` is false and the soft-error term is silenced. Real IB HCAs expose the standard `counters/` directory and are not affected.

2-CPU systems.

The IRQ skew effective threshold reaches 95% ($100/2 + 45$), making it nearly impossible to trigger the IRQ skew score term.

Throughput floor.

The $0.5\times$ floor on `ThroughputDropRule` means a legitimate workload reduction to 30% of baseline registers as a “drop” until the window passes.

12.3 Missing safeguards

- The stuck-state watchdog (200 windows) only logs; it does not auto-reset.
- No per-provider timeout in capability collection.
- No torn-read detection for sysfs counters.
- No 32-bit counter wrap detection.

13 Summary of Actual Behavior

This section describes what the engine actually does under specific scenarios—no generalization, just observable behavior.

Degrading IB cable with rising symbol errors. On a real IB fabric, a failing cable produces `symbol_error` and `link_error_recovery` deltas. `link_error_recovery` > 0 adds 0.35 to the composite score immediately, and `RdmaLinkDegradation` fires at Critical severity. The coverage-weighted floor pushes raw to ≥ 0.55 . State reaches Critical after 2–4 windows (~ 6 – 12 s). Remains Critical until errors stop for ~ 54 s.

Sustained packet loss on Soft-RoCE. Soft-error rate $\geq 5\%$ triggers `RdmaLinkDegradation`’s absolute ceiling every window. The composite raw score reaches $\sim 0.55+$ via the soft-error term plus the coverage-weighted severity floor. Behavior is otherwise identical to the IB case above.

Brief spike (5 windows) then perfect quiet. State reaches Critical after 4 windows of evidence. After the spike ends, peak-hold keeps effective score above `critical_exit` for ~ 5 windows. Total time from spike end to Healthy: ~ 17 additional windows (~ 51 s).

Slow drift over 10 minutes. `RdmaLinkDegradation`’s clamped EWMA prevents adaptation. The absolute ceiling fires once the rate exceeds 5%. `RisingErrorTrendRule` fires after 3 monotonically increasing windows.

Healthy node adjacent to a failing cable. No local hard errors. `port_xmit_wait` may rise due to upstream credit stalls. `CongestionSpreadRule` fires when z-score ≥ 3 . The composite score receives a 0.10 “victim buffer” boost.

Workload stops (no traffic, no errors). Carry-forward decays raw at $0.9\times/\text{window}$. Score decreases over ~ 7 windows. State machine de-escalates on the smoothed schedule.

All fabric capabilities are Absent. Rules still fire and emit alerts, but the coverage-weighted floor is zero. The raw score depends entirely on the `compute_health_score` formula. Detection is reduced but not eliminated.

14 eBPF Kernel Probes

This section documents the in-kernel instrumentation that produces the raw data the detection engine evaluates.

14.1 Probe programs

The eBPF object (`argus-ebpf/src/`) contains four source files, each implementing probes for a specific subsystem:

14.2 BPF maps

All data flows from kernel to userspace through per-CPU array maps (no ring buffer for the hot path). This keeps probe overhead under $5\mu\text{s}$.

File	Programs	Kernel attachment
slab.rs	trace_kmem_cache_alloc, trace_kmem_cache_free, kprobe_slab_alloc_entry, kretprobe_slab_alloc	Tracepoints kmem/kmem_cache_alloc, kmem/kmem_cache_free; kprobe/kretprobe on
interrupts.rs	trace_irq_handler_entry, trace_napi_poll	Tracepoints irq/irq_handler_entry, napi/napi_poll
rdma_jitter.rs	kprobe_wr_submit, kprobe_cq_poll_entry, kretprobe_cq_poll	Kprobes resolved from /proc/kallsyms at runtime

Table 4: eBPF probe programs and their kernel attachments.

Map name	Type	Contents
IRQ_COUNTS	PerCpuArray<u64>	1 slot per CPU: cumulative IRQ handler entry count
SLAB_STATS	PerCpuArray<[u64;6]>	[alloc_count, free_count, total_bytes_req, total_bytes_alloc, total_latency_ns, max_latency_ns]
SLAB_ALLOC_SCRATCH	LruHashMap<u64,u64>	Key: pid_tgid; value: entry timestamp for latency pairing. Max 256 entries.
NAPI_STATS	PerCpuArray<[u64;3]>	[poll_count, total_work, total_budget]
WR.TIMESTAMPS	LruHashMap<u64,u64>	Key: QP number; value: WR submit timestamp. Max 4096 entries.
CQ_JITTER_STATS	PerCpuArray<[u64;4]>	[completion_count, total_latency_ns, max_latency_ns, stall_count] where stall threshold = 50 μ s
QP.OWNERS	LruHashMap<u32,u32>	QP $\rightarrow$ TGID. Max 4096 entries.
CQ_POLL_SCRATCH	LruHashMap<u64,u64>	Key: pid_tgid; value: wc pointer. Max 256 entries.
LRU_MISSES	PerCpuArray<[u64;2]>	[cq_lru_misses, slab_lru_misses]. Counts when LRU lookup fails (eviction indicator).
OFFSETS	Array<u32>	8 entries: tracepoint field offsets and BTF struct offsets populated by userspace at load time.

Table 5: All BPF maps declared in `argus-ebpf/src/main.rs`.

14.3 CQ jitter measurement path

The CQ latency measurement works by correlating work request (WR) submission timestamps with completion queue (CQ) poll return values:

1. **WR submit kprobe:** Reads `qp_num` from the `ib_qp` struct (using BTF offsets from `OFFSETS`), stores (`qp_num  $\rightarrow$  ktime_ns`) in `WR.TIMESTAMPS` and (`qp_num  $\rightarrow$  pid`) in `QP.OWNERS`.
2. **CQ poll entry kprobe:** Saves the `ib_wc` pointer in `CQ_POLL_SCRATCH`.
3. **CQ poll kretprobe:** Iterates up to `min(num_completions, 8)` completions with stride 80 bytes. For each, reads `wc[i].qp  $\rightarrow$  qp_num`, looks up the submit timestamp in `WR.TIMESTAMPS`, computes `latency = now - submit_ts`, and updates `CQ_JITTER_STATS`. If the WR timestamp lookup fails (LRU eviction), increments `LRU_MISSES[0]`.

Caution

The LRU maps have fixed capacity. Under high QP counts (> 4096 active queue pairs), WR.TIMESTAMPS entries are evicted before CQ poll, causing latency measurements to be lost. The `argus_ebpf_lru_evictions{map=cq}` Prometheus counter tracks this. If eviction rates are high, the CQ jitter metrics become unreliable.

15 Hardware Counter Collection

15.1 Counter types

The agent reads 17 distinct counter types from `/sys/class/infiniband/*/ports/*/`. Each is identified by a discriminant byte in the aggregator:

ID	Counter	sysfs path suffix
0	<code>symbol_error</code>	<code>counters/symbol_error_count</code>
1	<code>link_downed</code>	<code>counters/link_downed</code>
2	<code>port_rcv_errors</code>	<code>counters/port_rcv_errors</code>
3	<code>port_xmit_discards</code>	<code>counters/port_xmit_discards</code>
4	<code>port_rcv_data</code>	<code>counters/port_rcv_data</code>
5	<code>port_xmit_data</code>	<code>counters/port_xmit_data</code>
6	<code>port_rcv_remote_phys</code>	<code>counters/port_rcv_remote_physical_errors</code>
7	<code>local_link_integrity</code>	<code>counters/local_link_integrity_errors</code>
8	<code>excessive_buffer_overnrun</code>	<code>counters/excessive_buffer_overnrun_errors</code>
9	<code>hw_rcv_pkts</code>	<code>hw_counters/rx_read_requests</code>
10	<code>hw_xmit_pkts</code>	<code>hw_counters/tx_write_requests</code>
11	<code>rx_duplicate_request</code>	<code>hw_counters/duplicate_request</code>
12	<code>rx_seq_error</code>	<code>hw_counters/seq_error</code>
13	<code>rx_retry_exceeded</code>	<code>hw_counters/retry_exceeded_err</code>
14	<code>rx_send_error</code>	<code>hw_counters/send_err</code>
15	<code>link_error_recovery</code>	<code>counters/link_error_recovery</code>
16	<code>port_xmit_wait</code>	<code>counters/port_xmit_wait</code>

Throughput note. IB `port_rcv_data` and `port_xmit_data` report values in units of 4 bytes (32-bit words). The Prometheus metrics multiply by 4 to report actual bytes: `argus_ib_throughput_rcv_bytes` = `port_rcv_data_delta` × 4.

Delta accumulation. Deltas from multiple ports on the same device are summed into a single `IbCounterDeltas` struct. This means on a dual-port HCA, error deltas reflect the aggregate across both ports.

15.2 Derived metrics

The `IbCounterDeltas` struct provides several computed accessors:

- `total_hard_error_delta()`: symbol + rcv errors + remote physical + link integrity + buffer overrun
- `total_soft_error_delta()`: RXE duplicate + seq error + retry exceeded + send error
- `throughput_bytes()`: (`rcv_data` + `xmit_data`) × 4
- `throughput_pkts()`: rcv + xmit packet counts

- `has_traffic()`: `bytes > 0` or `pkts > 0`

16 Capability System

16.1 Design rationale

Different RDMA hardware exposes different observability surfaces. An IB HCA with full sysfs counters, a RoCE NIC with ethtool stats, and a Soft-RoCE device with minimal `hw_counters/` each require different backends to extract the same logical signal. The capability system abstracts this: each logical signal (e.g., “link errors”) can have multiple backend implementations, and the best available one is selected at startup.

16.2 Capabilities

Each capability represents a logical signal type:

Capability	What it measures
<code>LinkErrors</code>	Error counters (symbol, rcv, link integrity, etc.)
<code>Throughput</code>	Byte/packet throughput rate
<code>CompletionLatency</code>	CQ completion latency distribution (p50–p999)
<code>RetransmitSignal</code>	Retransmission rate or indicators
<code>PfcPause</code>	PFC (Priority Flow Control) pause frame counts
<code>CreditStall</code>	IB credit stalls (<code>port_xmit_wait</code>)
<code>QpAttribution</code>	Map QP numbers to processes
<code>PeerLiveness</code>	Remote node health (probe-based)
<code>EcnSignal</code>	ECN marking rates
<code>CnpSignal</code>	Congestion Notification Packet rates

16.3 Quality levels and coverage grading

Each provider declares a quality level based on how trustworthy its data is:

Quality	Weight	Meaning
High	1.0	Direct hardware counters
Medium	0.6	Derived or sampled data
Low	0.3	Heuristic or inferred
Absent	0.0	Not available on this hardware

At startup, the `CapabilityRegistry` probes all candidate providers and selects the highest-quality available backend for each capability. A `CoverageReport` is assembled with an overall grade:

Grade A All four critical capabilities (`LinkErrors`, `Throughput`, `CompletionLatency`, `RetransmitSignal`) at **High** quality.

Grade B All critical capabilities at **Medium** or above.

Grade C At least one critical capability is non-**Absent**.

Grade F All critical capabilities are **Absent** (e.g., no IB hardware present).

16.4 Provider backends

The system ships with 17 candidate providers:

- SysfsLinkErrors
- InferredLinkErrors
- SysfsThroughput
- EbpfCqLatency
- ThroughputProxyLatency
- EbpfRetransmit
- MadRetransmit
- SysfsRetransmit
- InferredRetransmit
- EbpfQpAttribution
- RdmaCoreQpAttribution
- SysfsRxeQpAttribution
- EthtoolPfc
- InferredPfc
- EthtoolEcn
- EthtoolCnp
- PeerLiveness

17 Scheduler Integration

17.1 Architecture

The scheduler subsystem bridges ARGUS health verdicts and workload schedulers. It uses a reconciliation loop: each cycle, the reconciler compares the *desired* node state (derived from ARGUS health) against the *observed* state (queried from the scheduler backend) and takes action to converge them.

17.2 State model

Desired states	Observed states
Available	Available
Draining	Draining (jobs still running)
HeldByOperator	Drained (empty)
	Down
	Unknown

17.3 Policy

The `SchedulingPolicy` maps health states to desired states:

- **Healthy** → **Available** (unless operator hold is set)
- **Degraded** / **Recovering** → **Draining** only if `drain_on_degraded` is enabled; otherwise, keeps current desired state
- **Critical** → **Draining** always

Operator holds (`HeldByOperator`) are never auto-cleared. They require explicit release via `/scheduler/release` or the `argus-scheduler release` CLI.

17.4 Safety mechanisms

Advisory lock

The reconciler holds an `flock` on `/var/run/argus/scheduler.lock` to prevent concurrent reconciliation from multiple agent instances.

Drain rate limit

Default maximum 3 drains per hour. Rejections are counted in `argus_scheduler_drain_rejected_total`.

Resume cooldown

Default 60 seconds after drain before a resume is allowed.

Contested resume detection

If a node was drained by ARGUS but appeared resumed by an external actor (e.g., operator ran `scontrol update`), ARGUS enters a 300-second cooldown before re-draining.

External drain detection

If the node is drained but the reason does not start with `ARGUS:`, the reconciler infers an external operator hold and sets `HeldByOperator`.

Persisted state

Scheduler state survives restarts via `/var/lib/argus/scheduler-state.json` (atomic write with `tmp+rename`).

Audit log

All scheduler actions are appended to `/var/lib/argus/scheduler-audit.jsonl` (JSON lines format).

17.5 SLURM backend

The SLURM backend interacts via `scontrol` with sanitized environment (`env_clear()`, minimal `PATH/HOME`):

- **Drain:** `scontrol update NodeName=node State=DRAIN Reason=ARGUS:reason`
- **Resume:** `scontrol update NodeName=node State=RESUME`
- **Ownership:** Reason prefix `ARGUS:` identifies self-managed drains
- **Timeout:** 10 seconds per `scontrol` invocation

18 Autonomous Actions

Beyond scheduler integration, ARGUS can take autonomous actions when health states change.

18.1 Action types

Log action

Always enabled. Logs every alert via `tracing::warn!`.

Webhook

Enabled via `ARGUS_ACTION_WEBHOOK`. POST JSON payload containing `source`, `severity`, `kind`, `message`, `affected`, and `timestamp_ns`. Global timeout: 5 seconds. Rejects HTTP status ≥ 400 .

Port disable

Enabled via `ARGUS_ACTION_PORT_DISABLE` (dangerous). Only fires on **Critical** severity with `RdmaLinkDegradation` alert and `link_downed_delta > 0`. Writes "0" to sysfs `admin_state`. Refuses to act if multiple IB ports exist (ambiguity guard).

18.2 Rate limiting

Actions are rate-limited to one dispatch per 60 seconds. An internal audit log with capacity for 10,000 entries tracks all dispatched actions.

19 Prometheus Metrics Reference

The following table lists all Prometheus metrics exposed on `/metrics`. The “`_total`” suffix is added automatically by OpenMetrics encoding for counter types.

19.1 Health and detection

Metric	Type	Description
<code>argus_health_state</code>	Gauge	0=H, 1=D, 2=C, 3=R
<code>argus_health_score_millis</code>	Gauge	Composite score $\times 1000$
<code>argus_health_score_raw_millis</code>	Gauge	Pre-smoothing score
<code>argus_health_score_effective_millis</code>	Gauge	Post-smoothing score
<code>argus_health_score_ewma_millis</code>	Gauge	EWMA track
<code>argus_health_score_peak_hold_millis</code>	Gauge	Peak-hold track
<code>argus_events</code>	Gauge	Cumulative event count
<code>argus_alerts</code>	Counter	Labels: <code>kind</code> , <code>severity</code>
<code>argus_state_transitions</code>	Counter	Labels: <code>from</code> , <code>to</code>

19.2 eBPF subsystem metrics

Metric	Type	Description
<code>argus_irq{cpu}</code>	Counter	Per-CPU IRQ count
<code>argus_irq_imbalance_pct</code>	Gauge	Dominant CPU share
<code>argus_irq_dominant_cpu</code>	Gauge	CPU index
<code>argus_irq_total_rate</code>	Gauge	Window IRQ total
<code>argus_slab_alloc</code>	Counter	Cumulative slab allocations
<code>argus_slab_avg_latency_ns</code>	Gauge	Window average
<code>argus_slab_max_latency_ns</code>	Gauge	Window maximum
<code>argus_cq_completions</code>	Counter	CQ completions from eBPF
<code>argus_cq_avg_latency_ns</code>	Gauge	Window average
<code>argus_cq_max_latency_ns</code>	Gauge	Window maximum
<code>argus_cq_stalls</code>	Counter	Completions $> 50\mu s$
<code>argus_cq_p99_latency_ns</code>	Gauge	Estimated p99
<code>argus_napi_utilization_pct</code>	Gauge	work/budget $\times 100$
<code>argus_ebpf_lru_evictions{map}</code>	Counter	LRU map eviction count

19.3 IB hardware counter metrics

All IB metrics carry labels `device` and `port`:

Metric	Source
argus_ib_error_delta_total	Sum of all error deltas
argus_ib_symbol_errors_delta	symbol_error_delta
argus_ib_link_downed_delta	link_downed_delta
argus_ib_link_error_recovery_delta	link_error_recovery_delta
argus_ib_port_rcv_errors_delta	port_rcv_errors_delta
argus_ib_port_xmit_discards_delta	port_xmit_discards_delta
argus_ib_port_xmit_wait_delta	port_xmit_wait_delta
argus_ib_throughput_rcv_bytes	rcv_data $\times$ 4
argus_ib_throughput_xmit_bytes	xmit_data $\times$ 4
argus_ib_throughput_rcv_pkts	hw_rcv_pkts_delta
argus_ib_throughput_xmit_pkts	hw_xmit_pkts_delta
argus_ib_rxe_*.delta (4 metrics)	RXE soft-error counters

19.4 Capability and timescale metrics

Metric	Type	Description
argus_capability_coverage{cap,backend}	Gauge	Quality as integer
argus_capability_grade	Gauge	A=3, B=2, C=1, F=0
argus_sample_contribution_millis	Gauge	Fusion Δ
argus_cq_latency_p{50,95,99,999}_ns	Gauge	DDSketch quantiles
argus_timescale_state{timescale}	Gauge	Per-track state
argus_timescale_score_millis{timescale}	Gauge	Per-track score
argus_burst_classification{class}	Gauge	1=active, 0=not

19.5 Scheduler metrics

Metric	Type	Description
argus_scheduler_desired_state	Gauge	0=avail, 1=draining, 2=held
argus_scheduler_observed_state	Gauge	0-4 mapping
argus_scheduler_drain{mode}	Counter	Drain events
argus_scheduler_resume{mode}	Counter	Resume events
argus_scheduler_errors{op}	Counter	Backend errors
argus_scheduler_last_action_timestamp	Gauge	Unix seconds
argus_scheduler_drain_duration_seconds	Gauge	Time in drained state
argus_scheduler_drain_rejected_total	Counter	Rate-limit rejections

20 Security Architecture

ARGUS implements defense-in-depth through multiple security layers.

20.1 Privilege model

The agent requires elevated privileges to load eBPF programs and read kernel symbols. After initialization, it drops all capabilities except `CAP_BPF` (needed for ongoing BPF map reads on kernels < 5.19).

1. At startup: full capability set (from systemd `AmbientCapabilities`).
2. After eBPF load: `PR_SET_NO_NEW_PRIVS` is set, then all capabilities except `CAP_BPF` are dropped from both `Effective` and `Permitted` sets (loop over caps 0–63).
3. `umask(0o077)` is pinned at process start to prevent world-readable file creation.

4. PID file uses `O_NOFOLLOW` to prevent symlink attacks.

20.2 Systemd hardening

The production `argusd.service` unit applies 30+ hardening directives:

- `ProtectSystem=strict`, `ProtectHome=yes`, `PrivateTmp=yes`
- `MemoryDenyWriteExecute=yes` (prevents JIT/ROP gadgets)
- `SystemCallFilter`: layered allowlist (`@system-service`, `@io-event`, `@network-io`) with explicit denylists (`@mount`, `@swap`, `@reboot`, `ptrace`, `process_vm_readv/writev`)
- `ProtectProc=invisible`, `ProcSubset=pid` (hides other processes)
- `RestrictAddressFamilies=AF_INET AF_INET6 AF_UNIX AF_NETLINK`
- `NoNewPrivileges=yes`, `SecureBits=noroot-locked`
- `CapabilityBoundingSet`: only `CAP_BPF`, `CAP_PERFMON`, `CAP_SYSLOG`, `CAP_DAC_READ_SEARCH`
- Resource limits: `CPUQuota=5%`, `MemoryMax=256M`, `TasksMax=64`

20.3 eBPF integrity

Before loading the eBPF object, the agent verifies its SHA-256 hash against a known-good value from `/etc/argus/ebpf.sha256` or the `ARGUS_EBPF_HASH` environment variable. Hash mismatch aborts startup.

20.4 Metrics endpoint security

- Optional TLS with certificate hot-reloading via `SIGHUP`
- Bearer token authentication with constant-time comparison (prevents timing side-channels)
- Default bind to `127.0.0.1:9100` (loopback only)
- Connection semaphore and idle timeout for DoS protection
- Config file (`argusd.conf`) installed mode `0640` to protect token values from local users

21 Configuration Reference

21.1 Detection tuning parameters

These are configurable via the `[detection]` section of the TOML config file:

Parameter	Default	Effect
<code>irq_skew_threshold_pct</code>	70.0	Base threshold for IRQ affinity skew rule
<code>rdma_spike_factor</code>	5.0	Multiplier for RDMA latency spike detection
<code>rdma_baseline_latency_ns</code>	2000	Expected normal CQ completion latency
<code>slab_pressure_min_allocs</code>	100	Minimum allocs before slab pressure rule considers firing
<code>slab_pressure_alloc_rate_threshold</code>	5000	Alloc count above which correlation with IB errors is checked

21.2 State machine parameters

Configurable via `[detection.state_machine]` in TOML:

Parameter	Default	Effect
<code>degrade_enter</code>	0.30	Score threshold to enter Degraded
<code>degrade_exit</code>	0.10	Score threshold to exit Degraded
<code>critical_enter</code>	0.55	Score threshold to enter Critical
<code>critical_exit</code>	0.30	Score threshold to exit Critical
<code>enter_windows</code>	2	Consecutive windows to escalate
<code>exit_windows</code>	5	Consecutive windows to de-escalate
<code>recover_windows</code>	3	Mandatory hold in Recovering
<code>max_hold_windows</code>	200	Stuck-state watchdog

21.3 Non-configurable rule parameters

The following rule-internal parameters are hard-coded and cannot be changed without modifying source:

Rule	Parameter	Value
<code>RdmaLinkDegradation</code>	z-score threshold	3.0
<code>RdmaLinkDegradation</code>	error budget (per window)	5
<code>RdmaLinkDegradation</code>	absolute error rate ceiling	5%
<code>RdmaLinkDegradation</code>	cooldown windows	3
<code>RisingErrorTrend</code>	min consecutive windows	3
<code>LatencyDrift</code>	z-score threshold	3.0
<code>ThroughputDrop</code>	drop threshold	50%
<code>ThroughputDrop</code>	EWMA floor factor	0.5×
<code>NapiSaturation</code>	utilization threshold	85%
<code>NapiSaturation</code>	critical threshold	95%
<code>CqJitter</code>	stall threshold	50 μ s
<code>CqJitter</code>	critical stall count	> 10
<code>CqJitter</code>	critical max latency	> 1 ms
<code>CongestionSpread</code>	z-score threshold	3.0
<code>PcieBottleneck</code>	min CQ stalls	5
<code>PcieBottleneck</code>	min slab allocs	1000

21.4 Environment variables

All CLI flags can be set via environment variables with the `ARGUS_` prefix (read by `clap` directly—no shell interpretation):

Variable	Maps to
ARGUS_MODE	--mode (live/mock/replay)
ARGUS_EBPF_PATH	--ebpf-path
ARGUS_EBPF_HASH	--ebpf-hash
ARGUS_METRICS_ADDR	--metrics-addr
ARGUS_LOG_LEVEL	--log-level
ARGUS_WINDOW_SECS	--window-secs (1-300)
ARGUS_SCHEDULER	--scheduler (slurm/noop)
ARGUS_SCHEDULER_DRY_RUN	--scheduler-dry-run
ARGUS_SECCOMP	--seccomp
ARGUS_TLS_CERT	--tls-cert
ARGUS_TLS_KEY	--tls-key
ARGUS_METRICS_TOKEN	--metrics-token
ARGUS_ACTION_WEBHOOK	--action-webhook
ARGUS_READ_ONLY	--read-only
ARGUS_CONFIG	--config (TOML path)

Precedence: CLI flags > env vars > TOML config > compiled defaults.

Closing Notes

Why it behaves this way. Every default parameter—peak-hold decay ($\rho = 0.85$), exit dwell (5 windows), recovery hold (3 windows), critical exit (0.30), carry-forward (0.9)—is tuned for **automation safety**. The design philosophy: it is much worse to return a faulty node to production than to keep a healthy node idle for an extra minute.

Where it fails. (i) On devices with incomplete sysfs counter layouts (primarily Soft-RoCE test environments), the soft-error score term is silenced; real IB HCAs are unaffected. (ii) When capability providers report **Absent**, the coverage floor drops to zero and rules alone may not push the score high enough. (iii) The multi-timescale evaluator is published but does not govern the public verdict, which can confuse operators reading Grafana.

In one sentence. The engine runs 11 rules every 3-second window, fuses capability samples, smooths the result through EWMA + peak-hold, and feeds the smoothed score into a 4-state asymmetric-hysteresis state machine that escalates in ~ 12 s and recovers in ~ 54 s.